In my 6+ years of experience as an AWS DevOps and Kubernetes Specialist, I have extensively worked with various AWS services that are crucial for cloud infrastructure management and deployment processes. My proficiency includes services such as EC2, S3, RDS, Lambda, and CloudFormation, which I have utilized to design and implement scalable and reliable cloud architectures. For instance, in my recent role at Wipro, I was responsible for migrating NAS systems to AWS, which involved setting up EC2 instances for compute resources and S3 for scalable storage solutions. I also leveraged CloudFormation to automate the infrastructure provisioning process, ensuring consistency and reducing deployment time significantly.

Additionally, I have hands-on experience with AWS monitoring tools like CloudWatch, which I used to track system performance and set up alarms for resource utilization. This proactive monitoring allowed me to optimize costs and ensure high availability of applications. My familiarity with IAM roles and security groups has also enabled me to implement security best practices, ensuring that our cloud environments are compliant and secure. Overall, my experience with AWS has equipped me with the skills necessary to design, implement, and manage cloud infrastructure effectively, aligning perfectly with the requirements of this role.

I specialize in setting up and maintaining CI/CD pipelines using tools like **Jenkins, GitHub Actions, and Bamboo**, and have hands-on experience with **Docker and Kubernetes** for containerization and orchestration.Code quality and security are important to me — I've integrated tools like **SonarQube and Snyk** into pipelines to ensure clean, secure builds. I also set up monitoring solutions using **Prometheus, Grafana, and CloudWatch**, ensuring system reliability and fast issue detection.

Where are Nexus artifacts stored?
Typically under $data-dir/blobs in the file system.
What is Nexus Repository Manager, and why is it used?
Used to store, manage, and retrieve artifacts (e.g., Maven, npm, Docker) in a central place.
What are the types of repositories in Nexus?

- Hosted
- Proxy (remote)
- Group

1. **How does Jenkins integrate with Nexus?**

Jenkins pushes artifacts to Nexus using plugins (e.g., Maven Deploy, Nexus

Artifact Uploader).

1. **How does Jenkins integrate with Nexus?**

Jenkins pushes artifacts to Nexus using plugins (e.g., Maven Deploy, Nexus Artifact Uploader).

What is a *Code Smell* in SonarQube?

A **code smell** in SonarQube is a **maintainability issue** in the code — something that **doesn't necessarily cause bugs**, but **makes the code harder to understand, maintain, or extend**.

How do you integrate SonarQube with Jenkins or a CI/CD pipeline?

- Use the **SonarQube Scanner plugin**
- Pass sonar.projectKey, sonar.host.url, and sonar.login in the pipeline or Maven/Gradle config.

What is the Role of sonar-scanner in SonarQube?

The **sonar-scanner** is the **official command-line tool** provided by SonarQube to **analyze source code** and **send the results to the SonarQube server** for processing.

GitHub Authentication (OAuth)

Allows SSO with GitHub accounts.

✅ **Steps:**

1. Go to **Administration → ALM Integrations → GitHub**
2. Configure a new OAuth App in GitHub:
   - Go to GitHub → Settings → Developer settings → OAuth Apps
   - Set **Authorization callback URL**:
     bash
     CopyEdit

     http://<your-sonarqube-domain>/oauth2/callback/github
   -
3. Get:
   - Client ID
   - Client Secret
4. Paste into SonarQube GitHub config

What Are Unique Artifacts in Nexus?

Unique artifacts refer to files stored in Nexus (like .jar, .war, .zip, Docker images) that are **uniquely identified** by their:

- **Name**
- **Version**

- **Metadata (timestamp, commit ID, etc.)**

This uniqueness helps avoid collisions and supports versioning, rollback, and traceability.

Real-World Use Case	Artifact Type	Uniqueness Strategy
Maven builds in Jenkins	.jar, .war	Use SNAPSHOT or timestamped versions
Docker images	Images	Tag with commit hash, build ID, branch
Frontend zip files in raw repo	.zip, .tar.gz	Include version/ timestamp in file name
Multi-module microservices	All formats	Version per module, centralized in Nexus

CI/CD Workflow with Monitoring Tools Integrated

Stage	Tool(s) Involved	Monitoring Strategy
1. Code Commit	GitHub / GitLab	Webhook triggers CI; audit tracked in CloudTrail
2. Build & Test	Jenkins, Maven, SonarQube	SonarQube for code quality; Jenkins health & logs
3. Artifact Upload	Nexus	CloudWatch (EC2), disk usage monitored
4. Docker Image Build	Docker + Jenkins	Check for CVEs with Trivy or Clair; resource usage alert
5. Deployment	EKS (via Helm or kubectl)	Prometheus checks K8s resource usage, deployment success
6. Post-Deploy Test	Selenium, Postman, custom test scripts	Alert on failed health checks or latency issues
7. Production Monitoring	Prometheus, Grafana, ELK, Alertmanager	Track errors, CPU, memory, response times, logs

CloudWatch (AWS)

- Monitors EC2 (Jenkins, Nexus, SonarQube)
- Triggers SNS alerts if:
  - Disk usage > 90%
  - Jenkins agent goes offline
- Integrated with Lambda for auto-recovery

SonarQube

- Scans source code during Jenkins build
- If **Quality Gate fails**, pipeline halts
- Dashboards show bugs, vulnerabilities, code smells

End-to-End Pipeline Overview with bamboo

Dev commits code → Bitbucket PR → Bamboo detects commit → CI build
→ SonarQube scan → Artifact pushed to Nexus
→ Docker image build + push → Helm deployment to EKS
→ Slack/JIRA updated + monitoring starts (Prometheus/Grafana)

CI/CD Bamboo + DevOps Toolchain Summary

Stage	Tool	Purpose
Source Control	Bitbucket	Git, PR-based triggering
Build	Bamboo + Maven	Compile, test
Code Quality	SonarQube	Bugs, code smells, vulnerabilities
Artifact Store	Nexus	Stores .jar, .war, Docker
Containerization	Docker	Build images
Deployment	Helm + EKS	K8s-based rollout
Monitoring	Prometheus + Grafana	Post-deploy monitoring
Notification	Slack, JIRA	Real-time updates and traceability

Check Disk Usage and Alert

```
#!/bin/bash
THRESHOLD=80
USAGE=$(df / | grep / | awk '{print $5}' | sed 's/%//g')

if [ "$USAGE" -gt "$THRESHOLD" ]; then
  echo "Disk usage is above $THRESHOLD%! Current usage: $USAGE%" | mail
-s "Disk Alert" admin@example.com
fi
```

Log rotation

```
#!/bin/bash

LOG_FILE="/var/log/myapp.log"
BACKUP_FILE="/var/log/myapp.log.1"
```

```
# Check if log file exists
if [ -f "$LOG_FILE" ]; then
    # Move current log to backup
    mv "$LOG_FILE" "$BACKUP_FILE"
    echo "Rotated $LOG_FILE to $BACKUP_FILE"

    # Create new empty log file
    touch "$LOG_FILE"
    chmod 644 "$LOG_FILE"
else
    echo "Log file $LOG_FILE does not exist."
fi
```

Scrum Workflow Summary

1. Product Owner refines the **Product Backlog**
2. During **Sprint Planning**, the team selects work into the **Sprint Backlog**
3. The team collaborates daily via **Daily Stand-ups**
4. At the end of the Sprint:
    - Product is demoed in the **Sprint Review**
    - Team reflects in the **Retrospective**
5. Process repeats in the next Sprint

```
# delete the files older than 7 days
#!/bin/bash.

#dorectory containing the files
 log_dir = “/var/log/app.log”

# now find the .log file and delete the these

Find $log-dir -type f -name “*.log” -mtype +7 -exec rm -f {} \;

Echo “ [ )$data, +%h-%m-%s _% 0}
```

Explain a typical CI/CD pipeline you have implemented using Bamboo and Harness.
Answer:
In my previous project, the pipeline triggered a Bamboo build when code was pushed to Bitbucket. Bamboo executed unit tests, built the artifact, and stored it in Nexus. Harness picked up the artifact from Nexus and deployed it to the development environment automatically. Post deployment, Harness ran automated verification tests and, upon success, promoted the deployment to staging with manual approval before production deployment.

How do you handle failures during deployments?
Candidate: Harness supports automatic rollback triggered by deployment verification failures. Additionally, I use manual approval steps in sensitive environments like production. Logs and monitoring alerts help quickly identify issues, and I ensure detailed audit trails are available for postmortem analysis.

Difference between freestyle and pipeline jobs in Jenkins?

Freestyle Job	Pipeline Job
GUI-based	Code-defined (Groovy/DSL)
Limited customization	Fully customizable & reusable
Poor for complex flows	Best for multi-stage CI/CD

jenkins pipeline failure handling – real issue & fix?
Issue: docker push failed due to incorrect credentials
Fix: Added encrypted DockerHub credentials to Jenkins Credentials, used withCredentials() block in pipeline to authenticate securely. Also added retry logic.

CMD vs ENTRYPOINT in Docker?

CMD	ENTRYPOINT
Default command	Always executed
Can be overridden easily	Harder to override (unless --entrypoint is used)

Use ENTRYPOINT for scripts, CMD for default arguments.

What is Docker Compose and where have you used it?
Docker Compose allows running multi-container apps with a single YAML file. I used it in local development to spin up services like app, DB, and Redis together.

What are pods, deployments, and services in K8s?

- **Pod**: Basic unit of deployment (can have 1+ containers)
- **Deployment**: Manages pod lifecycle (rolling updates, scaling)
- **Service**: Exposes pods as a stable endpoint (ClusterIP, NodePort, LoadBalancer)

Pipeline Flow for Each Microservice

1. **Code Push to GitHub**
   ➜ Triggers CI pipeline
2. **CI Pipeline**:
   - Checkout code
   - Run unit tests

    - Static code analysis via **SonarQube**
    - Build Docker image
    - Push image to **Nexus or ECR**
3. **CD Pipeline**:
    - Update Helm chart with new image tag
    - Deploy to **dev** namespace in Kubernetes
    - Run automated smoke tests
    - On success, promote to **staging** and eventually **production**
4. **Rolling Update Strategy**:
    - Kubernetes Deployment object uses RollingUpdate strategy
    - New pods spun up before terminating old ones
    - Achieved **zero downtime**